

## 2

# FUNDAMENTAL CONCEPTS

This chapter introduces a range of concepts related to Linux system programming. It is intended for readers who have worked primarily with other operating systems, or who have only limited experience with Linux or another UNIX implementation.

## 2.1 The Core Operating System: The Kernel

The term *operating system* is commonly used with two different meanings:

- To denote the entire package consisting of the central software managing a computer’s resources and all of the accompanying standard software tools, such as command-line interpreters, graphical user interfaces, file utilities, and editors.
- More narrowly, to refer to the central software that manages and allocates computer resources (i.e., the CPU, RAM, and devices).

The term *kernel* is often used as a synonym for the second meaning, and it is with this meaning of the term *operating system* that we are concerned in this book.

Although it is possible to run programs on a computer without a kernel, the presence of a kernel greatly simplifies the writing and use of other programs, and increases the power and flexibility available to programmers. The kernel does this by providing a software layer to manage the limited resources of a computer.

The Linux kernel executable typically resides at the pathname `/boot/vmlinuz`, or something similar. The derivation of this filename is historical. On early UNIX implementations, the kernel was called `unix`. Later UNIX implementations, which implemented virtual memory, renamed the kernel as `vmunix`. On Linux, the filename mirrors the system name, with the `z` replacing the final `x` to signify that the kernel is a compressed executable.

### Tasks performed by the kernel

Among other things, the kernel performs the following tasks:

- *Process scheduling*: A computer has one or more central processing units (CPUs), which execute the instructions of programs. Like other UNIX systems, Linux is a *preemptive multitasking* operating system, *Multitasking* means that multiple processes (i.e., running programs) can simultaneously reside in memory and each may receive use of the CPU(s). *Preemptive* means that the rules governing which processes receive use of the CPU and for how long are determined by the kernel process scheduler (rather than by the processes themselves).
- *Memory management*: While computer memories are enormous by the standards of a decade or two ago, the size of software has also correspondingly grown, so that physical memory (RAM) remains a limited resource that the kernel must share among processes in an equitable and efficient fashion. Like most modern operating systems, Linux employs virtual memory management (Section 6.4), a technique that confers two main advantages:
  - Processes are isolated from one another and from the kernel, so that one process can’t read or modify the memory of another process or the kernel.
  - Only part of a process needs to be kept in memory, thereby lowering the memory requirements of each process and allowing more processes to be held in RAM simultaneously. This leads to better CPU utilization, since it increases the likelihood that, at any moment in time, there is at least one process that the CPU(s) can execute.
- *Provision of a file system*: The kernel provides a file system on disk, allowing files to be created, retrieved, updated, deleted, and so on.
- *Creation and termination of processes*: The kernel can load a new program into memory, providing it with the resources (e.g., CPU, memory, and access to files) that it needs in order to run. Such an instance of a running program is termed a *process*. Once a process has completed execution, the kernel ensures that the resources it uses are freed for subsequent reuse by later programs.
- *Access to devices*: The devices (mice, monitors, keyboards, disk and tape drives, and so on) attached to a computer allow communication of information between the computer and the outside world, permitting input, output, or both. The kernel provides programs with an interface that standardizes and simplifies access to devices, while at the same time arbitrating access by multiple processes to each device.
- *Networking*: The kernel transmits and receives network messages (packets) on behalf of user processes. This task includes routing of network packets to the target system.
- *Provision of a system call application programming interface (API)*: Processes can request the kernel to perform various tasks using kernel entry points known as *system calls*. The Linux system call API is the primary topic of this book. Section 3.1 details the steps that occur when a process performs a system call.

In addition to the above features, multiuser operating systems such as Linux generally provide users with the abstraction of a *virtual private computer*; that is, each user can log on to the system and operate largely independently of other users. For example, each user has their own disk storage space (home directory). In addition, users can run programs, each of which gets a share of the CPU and operates in its own virtual address space, and these programs can independently access devices and transfer information over the network. The kernel resolves potential conflicts in accessing hardware resources, so users and processes are generally unaware of the conflicts.

### Kernel mode and user mode

Modern processor architectures typically allow the CPU to operate in at least two different modes: *user mode* and *kernel mode* (sometimes also referred to as *supervisor mode*). Hardware instructions allow switching from one mode to the other. Correspondingly, areas of virtual memory can be marked as being part of *user space* or *kernel space*. When running in user mode, the CPU can access only memory that is marked as being in user space; attempts to access memory in kernel space result in a hardware exception. When running in kernel mode, the CPU can access both user and kernel memory space.

Certain operations can be performed only while the processor is operating in kernel mode. Examples include executing the halt instruction to stop the system, accessing the memory-management hardware, and initiating device I/O operations. By taking advantage of this hardware design to place the operating system in kernel space, operating system implementers can ensure that

user processes are not able to access the instructions and data structures of the kernel, or to perform operations that would adversely affect the operation of the system.

Process versus kernel views of the system

In many everyday programming tasks, we are accustomed to thinking about programming in a process-oriented way. However, when considering various topics covered later in this book, it can be useful to reorient our perspective to consider things from the kernel’s point of view. To make the contrast clear, we now consider how things look first from a process viewpoint and then from a kernel viewpoint.

A running system typically has numerous processes. For a process, many things happen asynchronously. An executing process doesn’t know when it will next time out, which other processes will then be scheduled for the CPU (and in what order), or when it will next be scheduled. The delivery of signals and the occurrence of interprocess communication events are mediated by the kernel, and can occur at any time for a process. Many things happen transparently for a process. A process doesn’t know where it is located in RAM or, in general, whether a particular part of its memory space is currently resident in memory or held in the swap area (a reserved area of disk space used to supplement the computer’s RAM). Similarly, a process doesn’t know where on the disk drive the files it accesses are being held; it simply refers to the files by name. A process operates in isolation; it can’t directly communicate with another process. A process can’t itself create a new process or even end its own existence. Finally, a process can’t communicate directly with the input and output devices attached to the computer.

By contrast, a running system has one kernel that knows and controls everything. The kernel facilitates the running of all processes on the system. The kernel decides which process will next obtain access to the CPU, when it will do so, and for how long. The kernel maintains data structures containing information about all running processes and updates these structures as processes are created, change state, and terminate. The kernel maintains all of the low-level data structures that enable the filenames used by programs to be translated into physical locations on the disk. The kernel also maintains data structures that map the virtual memory of each process into the physical memory of the computer and the swap area(s) on disk. All communication between processes is done via mechanisms provided by the kernel. In response to requests from processes, the kernel creates new processes and terminates existing processes. Lastly, the kernel (in particular, device drivers) performs all direct communication with input and output devices, transferring information to and from user processes as required.

Later in this book we’ll say things such as “a process can create another process,” “a process can create a pipe,” “a process can write data to a file,” and “a process can terminate by calling *exit()*.” Remember, however, that the kernel mediates all such actions, and these statements are just shorthand for “a process can *request that the kernel* create another process,” and so on.

Further information

Modern texts covering operating systems concepts and design, with particular reference to UNIX systems, include [Tanenbaum, 2007], [Tanenbaum & Woodhull, 2006], and [Vahalia, 1996], the last of these containing much detail on virtual memory architectures. [Goodheart & Cox, 1994] provide details on System V Release 4. [Maxwell, 1999] provides an annotated listing of selected parts of the Linux 2.2.5 kernel. [Lions, 1996] is a detailed exposition of the Sixth Edition UNIX source code that remains a useful introduction to UNIX operating system internals. [Bovet & Cesati, 2005] describes the implementation of the Linux 2.6 kernel.

2.2 The Shell

A *shell* is a special-purpose program designed to read commands typed by a user and execute appropriate programs in response to those commands. Such a program is sometimes known as a *command interpreter*.

The term *login shell* is used to denote the process that is created to run a shell when the user first logs in. Whereas on some operating systems the command interpreter is an integral part of the kernel, on UNIX systems, the shell is a user process. Many different shells exist, and different users (or, for that matter, a single user) on the same computer can simultaneously use different shells. A number of important shells have appeared over time:

- *Bourne shell (sh)*: This is the oldest of the widely used shells, and was written by Steve Bourne. It was the standard shell for Seventh Edition UNIX. The Bourne shell contains many of the features familiar in all shells: I/O redirection, pipelines, filename generation (globbing), variables, manipulation of environment variables, command substitution, background command execution, and functions. All later UNIX implementations include the Bourne shell in addition to any other shells they might provide.
- *C shell (csh)*: This shell was written by Bill Joy at the University of California at Berkeley. The name derives from the resemblance of many of the flow-control constructs of this shell to those of the C programming language. The C shell provided several useful interactive features unavailable in the Bourne shell, including command history, command-line editing, job control, and aliases. The C shell was not backward compatible with the Bourne shell. Although the standard interactive shell on BSD was the C shell, shell scripts (described in a moment) were usually written for the Bourne shell, so as to be portable across all UNIX implementations.
- *Korn shell (ksh)*: This shell was written as the successor to the Bourne shell by David Korn at AT&T Bell Laboratories. While maintaining backward compatibility with the Bourne shell, it also incorporated interactive features similar to those provided by the C shell.
- *Bourne again shell (bash)*: This shell is the GNU project’s reimplementation of the Bourne shell. It supplies interactive features similar to those available in the C and Korn shells. The principal authors of *bash* are Brian Fox and Chet Ramey. Bash is probably the most widely used shell on Linux. (On Linux, the Bourne shell, *sh*, is typically provided by *bash* emulating *sh* as closely as possible.)

POSIX.2-1992 specified a standard for the shell that was based on the then current version of the Korn shell. Nowadays, the Korn shell and *bash* both conform to POSIX, but provide a number of extensions to the standard, and many of these extensions differ between the two shells.

The shells are designed not merely for interactive use, but also for the interpretation of *shell scripts*, which are text files containing shell commands. For this purpose, each of the shells has the facilities typically associated with programming languages: variables, loop and conditional statements, I/O commands, and functions.

Each of the shells performs similar tasks, albeit with variations in syntax. Unless referring to the operation of a specific shell, we typically refer to “the shell,” with the understanding that all shells operate in the manner described. Most of the examples in this book that require a shell use *bash*, but, unless otherwise noted, the reader can assume these examples work the same way in other Bourne-type shells.

One of the distinguishing features of the I/O model on UNIX systems is the concept of *universality of I/O*. This means that the same system calls (*open()*, *read()*, *write()*, *close()*, and so on) are used to perform I/O on all types of files, including devices. (The kernel translates the application’s I/O requests into appropriate file-system or device-driver operations that perform I/O on the target file or device.) Thus, a program employing these system calls will work on any type of file.

The kernel essentially provides one file type: a sequential stream of bytes, which, in the case of disk files, disks, and tape devices, can be randomly accessed using the *lseek()* system call.

Many applications and libraries interpret the *newline* character (ASCII code 10 decimal, sometimes also known as *linefeed*) as terminating one line of text and commencing another. UNIX systems have no *end-of-file* character; the end of a file is detected by a read that returns no data.

**File descriptors**

The I/O system calls refer to open files using a *file descriptor*, a (usually small) nonnegative integer. A file descriptor is typically obtained by a call to *open()*, which takes a pathname argument specifying a file upon which I/O is to be performed.

Normally, a process inherits three open file descriptors when it is started by the shell: descriptor 0 is *standard input*, the file from which the process takes its input; descriptor 1 is *standard output*, the file to which the process writes its output; and descriptor 2 is *standard error*, the file to which the process writes error messages and notification of exceptional or abnormal conditions. In an interactive shell or program, these three descriptors are normally connected to the terminal. In the *stdio* library, these descriptors correspond to the file streams *stdin*, *stdout*, and *stderr*.

**The *stdio* library**

To perform file I/O, C programs typically employ I/O functions contained in the standard C library. This set of functions, referred to as the *stdio* library, includes *fopen()*, *fclose()*, *scanf()*, *printf()*, *fgets()*, *fputs()*, and so on. The *stdio* functions are layered on top of the I/O system calls (*open()*, *close()*, *read()*, *write()*, and so on).

We assume that the reader is already familiar with the C standard I/O (*stdio*) functions, and don’t cover them in this book. Further information on the *stdio* library can be found in [Kernighan & Ritchie, 1988], [Harbison & Steele, 2002], [Plauger, 1992], and [Stevens & Rago, 2005].

**2.6 Programs**

*Programs* normally exist in two forms. The first form is *source code*, human-readable text consisting of a series of statements written in a programming language such as C. To be executed, source code must be converted to the second form: binary machine-language instructions that the computer can understand. (This contrasts with a *script*, which is a text file containing commands to be directly processed by a program such as a shell or other command interpreter.) The two meanings of the term *program* are normally considered synonymous, since the step of compiling and linking converts source code into semantically equivalent binary machine code.

**Filters**

A *filter* is the name often applied to a program that reads its input from *stdin*, performs some transformation of that input, and writes the transformed data to *stdout*. Examples of filters include *cat*, *grep*, *tr*, *sort*, *wc*, *sed*, and *awk*.

**Command-line arguments**

In C, programs can access the *command-line arguments*, the words that are supplied on the command line when the program is run. To access the command-line arguments, the *main()* function of the program is declared as follows:

```
int main(int argc, char *argv[])
```

The *argc* variable contains the total number of command-line arguments, and the individual arguments are available as strings pointed to by members of the array *argv*. The first of these strings, *argv[0]*, identifies the name of the program itself.

**2.7 Processes**

Put most simply, a *process* is an instance of an executing program. When a program is executed, the kernel loads the code of the program into virtual memory, allocates space for program variables, and sets up kernel bookkeeping data structures to record various information (such as process ID, termination status, user IDs, and group IDs) about the process.

From a kernel point of view, processes are the entities among which the kernel must share the various resources of the computer. For resources that are limited, such as memory, the kernel initially allocates some amount of the resource to the process, and adjusts this allocation over the lifetime of the process in response to the demands of the process and the overall system demand for that resource. When the process terminates, all such resources are released for reuse by other processes. Other resources, such as the CPU and network bandwidth, are renewable, but must be shared equitably among all processes.

**Process memory layout**

A process is logically divided into the following parts, known as *segments*:

- *Text*: the instructions of the program.
- *Data*: the static variables used by the program.
- *Heap*: an area from which programs can dynamically allocate extra memory.
- *Stack*: a piece of memory that grows and shrinks as functions are called and return and that is used to allocate storage for local variables and function call linkage information.

**Process creation and program execution**

A process can create a new process using the *fork()* system call. The process that calls *fork()* is referred to as the *parent process*, and the new process is referred to as the *child process*. The kernel creates the child process by making a duplicate of the parent process. The child inherits copies of the parent’s data, stack, and heap segments, which it may then modify independently of the parent’s copies. (The program text, which is placed in memory marked as read-only, is shared by the two processes.)

The primary advantages of using threads are that they make it easy to share data (via global variables) between cooperating threads and that some algorithms transpose more naturally to a multithreaded implementation than to a multiprocess implementation. Furthermore, a multithreaded application can transparently take advantage of the possibilities for parallel processing on multiprocessor hardware.

## 2.13 Process Groups and Shell Job Control

Each program executed by the shell is started in a new process. For example, the shell creates three processes to execute the following pipeline of commands (which displays a list of files in the current working directory sorted by file size):

```
$ ls -l | sort -k5n | less
```

All major shells, except the Bourne shell, provide an interactive feature called *job control*, which allows the user to simultaneously execute and manipulate multiple commands or pipelines. In job-control shells, all of the processes in a pipeline are placed in a new *process group* or *job*. (In the simple case of a shell command line containing a single command, a new process group containing just a single process is created.) Each process in a process group has the same integer *process group identifier*, which is the same as the process ID of one of the processes in the group, termed the *process group leader*.

The kernel allows for various actions, notably the delivery of signals, to be performed on all members of a process group. Job-control shells use this feature to allow the user to suspend or resume all of the processes in a pipeline, as described in the next section.

## 2.14 Sessions, Controlling Terminals, and Controlling Processes

A *session* is a collection of process groups (jobs). All of the processes in a session have the same *session identifier*. A *session leader* is the process that created the session, and its process ID becomes the session ID.

Sessions are used mainly by job-control shells. All of the process groups created by a job-control shell belong to the same session as the shell, which is the session leader.

Sessions usually have an associated *controlling terminal*. The controlling terminal is established when the session leader process first opens a terminal device. For a session created by an interactive shell, this is the terminal at which the user logged in. A terminal may be the controlling terminal of at most one session.

As a consequence of opening the controlling terminal, the session leader becomes the *controlling process* for the terminal. The controlling process receives a `SIGHUP` signal if a terminal disconnect occurs (e.g., if the terminal window is closed).

At any point in time, one process group in a session is the *foreground process group* (*foreground job*), which may read input from the terminal and send output to it. If the user types the *interrupt* character (usually *Control-C*) or the *suspend* character (usually *Control-Z*) on the controlling terminal, then the terminal driver sends a signal that kills or suspends (i.e., stops) the foreground process group. A session can have any number of *background process groups* (*background jobs*), which are created by terminating a command with the ampersand (`&`) character.

Job-control shells provide commands for listing all jobs, sending signals to jobs, and moving jobs between the foreground and background.

## 2.15 Pseudoterminals

A *pseudoterminal* is a pair of connected virtual devices, known as the *master* and *slave*. This device pair provides an IPC channel allowing data to be transferred in both directions between the two devices.

The key point about a pseudoterminal is that the slave device provides an interface that behaves like a terminal, which makes it possible to connect a terminal-oriented program to the slave device and then use another program connected to the master device to drive the terminal-oriented program. Output written by the driver program undergoes the usual input processing performed by the terminal driver (for example, in the default mode, a carriage return is mapped to a newline) and is then passed as input to the terminal-oriented program connected to the slave. Anything that the terminal-oriented program writes to the slave is passed (after performing all of the usual terminal output processing) as input to the driver program. In other words, the driver program is performing the function normally performed by the user at a conventional terminal.

Pseudoterminals are used in a variety of applications, most notably in the implementation of terminal windows provided under an X Window System login and in applications providing network login services, such as *telnet* and *ssh*.

## 2.16 Date and Time

Two types of time are of interest to a process:

- *Real time* is measured either from some standard point (*calendar* time) or from some fixed point, typically the start, in the life of a process (*elapsed* or *wall clock* time). On UNIX systems, calendar time is measured in seconds since midnight on the morning of January 1, 1970, Coordinated Universal Time (usually abbreviated UTC), and coordinated on the base point for timezones defined by the longitudinal line passing through Greenwich, England. This date, which is close to the birth of the UNIX system, is referred to as the *Epoch*.
- *Process time*, also called *CPU time*, is the total amount of CPU time that a process has used since starting. CPU time is further divided into *system CPU time*, the time spent executing code in *kernel mode* (i.e., executing system calls and performing other kernel services on behalf of the process), and *user CPU time*, the time spent executing code in *user mode* (i.e., executing normal program code).

The *time* command displays the real time, the system CPU time, and user CPU time taken to execute the processes in a pipeline.

## 2.17 Client-Server Architecture

At various points in this book, we discuss the design and implementation of client-server applications.

A *client-server application* is one that is broken into two component processes:

- a *client*, which asks the server to carry out some *service* by sending it a request message; and
- a *server*, which examines the client’s request, performs appropriate actions, and then sends a response message back to the client.

Sometimes, the client and server may engage in an extended dialogue of requests and responses.

Typically, the client application interacts with a user, while the server application provides access to some shared resource. Commonly, there are multiple instances of client processes communicating with one or a few instances of the server process.

The client and server may reside on the same host computer or on separate hosts connected via a network. To communicate with one another, the client and server use the IPC mechanisms discussed in Section 2.10.

Servers may implement a variety of services, such as:

- providing access to a database or other shared information resource;
- providing access to a remote file across a network;
- encapsulating some business logic;
- providing access to a shared hardware resource (e.g., a printer); or
- serving web pages.

Encapsulating a service within a single server is useful for a number of reasons, such as the following:

- *Efficiency*: It may be cheaper to provide one instance of a resource (e.g., a printer) that is managed by a server than to provide the same resource locally on every computer.
- *Control, coordination, and security*: By holding a resource (especially an information resource) at a single location, the server can coordinate access to the resource (e.g., so that two clients don’t simultaneously update the same piece of information) or secure it so that it is made available to only selected clients.
- *Operation in a heterogeneous environment*: In a network, the various clients, and the server, can be running on different hardware and operating system platforms.

## 2.18 Realtime

*Realtime applications* are those that need to respond in a timely fashion to input. Frequently, such input comes from an external sensor or a specialized input device, and output takes the form of controlling some external hardware. Examples of applications with realtime response requirements include automated assembly lines, bank ATMs, and aircraft navigation systems.

Although many realtime applications require rapid responses to input, the defining factor is that the response is guaranteed to be delivered within a certain deadline time after the triggering event.

The provision of realtime responsiveness, especially where short response times are demanded, requires support from the underlying operating system. Most operating systems don’t natively provide such support because the requirements of realtime responsiveness can conflict with the requirements of multiuser timesharing operating systems. Traditional UNIX implementations are not realtime operating systems, although realtime variants have been devised. Realtime variants of Linux have also been created, and recent Linux kernels are moving toward full native support for realtime applications.

POSIX.1b defined a number of extensions to POSIX.1 for the support of real-time applications. These include asynchronous I/O, shared memory, memory-mapped files, memory locking, realtime clocks and timers, alternative scheduling policies, realtime signals, message queues, and semaphores. Even though they don’t strictly qualify as realtime, most UNIX implementations now support some or all of these extensions. (During the course of this book, we describe those features of POSIX.1b that are supported by Linux.)

In this book, we use the term *real time* to refer to the concept of calendar or elapsed time, and the term *realtime* to denote an operating system or application providing the type of responsiveness described in this section.

## 2.19 The /proc File System

Like several other UNIX implementations, Linux provides a `/proc` file system, which consists of a set of directories and files mounted under the `/proc` directory.

The `/proc` file system is a virtual file system that provides an interface to kernel data structures in a form that looks like files and directories on a file system. This provides an easy mechanism for viewing and changing various system attributes. In addition, a set of directories with names of the form `/proc/PID`, where *PID* is a process ID, allows us to view information about each process running on the system.

The contents of `/proc` files are generally in human-readable text form and can be parsed by shell scripts. A program can simply open and read from, or write to, the desired file. In most cases, a process must be privileged to modify the contents of files in the `/proc` directory.

As we describe various parts of the Linux programming interface, we’ll also describe the relevant `/proc` files. Section 12.1 provides further general information on this file system. The `/proc` file system is not specified by any standards, and the details that we describe are Linux-specific.

## 2.20 Summary

In this chapter, we surveyed a range of fundamental concepts related to Linux system programming. An understanding of these concepts should provide readers with limited experience on Linux or UNIX with enough background to begin learning system programming.

# 62

## TERMINALS

Historically, users accessed a UNIX system using a terminal connected via a serial line (an RS-232 connection). Terminals were cathode ray tubes (CRTs) capable of displaying characters and, in some cases, primitive graphics. Typically, CRTs provided a monochrome display of 24 lines by 80 columns. By today’s standards, these CRTs were small and expensive. In even earlier times, terminals were sometimes hard-copy teletype devices. Serial lines were also used to connect other devices, such as printers and modems, to a computer or to connect one computer to another.

On early UNIX systems, the terminal lines connected to the system were represented by character devices with names of the form `/dev/ttyn`. (On Linux, the `/dev/ttyn` devices are the virtual consoles on the system.) It is common to see the abbreviation *tty* (derived from teletype) as a shorthand for *terminal*.

Especially during the early years of UNIX, terminal devices were not standardized, which meant that different character sequences were required to perform operations such as moving the cursor to the beginning of the line or the top of the screen. (Eventually, some vendor implementations of such *escape sequences*—for example, Digital’s VT-100—became de facto and, ultimately, ANSI standards, but a wide variety of terminal types continued to exist.) This lack of standardization meant that it was difficult to write portable programs that used terminal features. The *vi* editor was an early example of a program with such requirements. The *termcap* and *terminfo* databases (described in [Strang et al., 1988]), which tabulate how to perform various screen-control operations for a wide variety of terminal types, and the *curses* library ([Strang, 1986]) were developed in response to this lack of standardization.

It is no longer common to see a conventional terminal. The usual interface to modern UNIX systems is an X Window System window manager on a high-performance bit-mapped graphical monitor. (An old-style terminal provided functionality roughly equivalent to a single terminal window—an *xterm* or similar—in an X Window System. The fact that the user of such a terminal had only a single “window” to the system was the driving force behind the development of the job-control facilities described in Section 34.7.) Similarly, many devices (e.g., printers) that were once connected directly to a computer are now often intelligent devices connected via a network.

All of the above is a preamble to saying that the need to program terminal devices is less frequent than it used to be. Therefore, this chapter focuses on the aspects of terminal programming that are particularly relevant to software terminal emulators (i.e., *xterm* and similar). It gives only brief coverage to serial lines; references for further information about serial programming are provided at the end of this chapter.

### 62.1 Overview

Both a conventional terminal and a terminal emulator have an associated terminal driver that handles input and output on the device. (In the case of a terminal emulator, the device is a pseudoterminal. We describe pseudoterminals in Chapter 64.) Various aspects of the operation of the terminal driver can be controlled using the functions described in this chapter.

When performing input, the driver is capable of operating in either of the following modes:

- *Canonical mode*: In this mode, terminal input is processed line by line, and line editing is enabled. Lines are terminated by a newline character, generated when the user presses the *Enter* key. A *read()* from the terminal returns only when a complete line of input is available, and returns at most one line. (If the *read()* requests fewer bytes than are available in the current line, then the remaining bytes are available for the next *read()*.) This is the default input mode.
- *Noncanonical mode*: Terminal input is not gathered into lines. Programs such as *vi*, *more*, and *less* place the terminal in noncanonical mode so that they can read single characters without the user needing to press the *Enter* key.

The terminal driver also interprets a range of special characters, such as the *interrupt* character (normally *Control-C*) and the *end-of-file* character (normally *Control-D*). Such interpretation may result in a signal being generated for the foreground process group or some type of input condition occurring for a program reading from the terminal. Programs that place the terminal in noncanonical mode typically also disable processing of some or all of these special characters.

A terminal driver operates two queues (Figure 62-1): one for input characters transmitted from the terminal device to the reading process(es) and the other for output characters transmitted from processes to the terminal. If terminal echoing is enabled, then the terminal driver automatically appends a copy of any input character to the end of the output queue, so that input characters are also output on the terminal.

SUSv3 specifies the limit `MAX_INPUT`, which an implementation can use to indicate the maximum length of the terminal’s input queue. A related limit, `MAX_CANON`, defines the maximum number of bytes in a line of input in canonical mode. On Linux, *pathconf()* returns the value 255 for both of these limits. However, neither of these limits is actually employed by the kernel, which simply imposes a limit of 4096 bytes on the input queue. A corresponding limit on the size of the output queue also exists. However, applications don’t need to be concerned with this, since, if a process produces output faster than the terminal driver can handle it, the kernel suspends execution of the writing process until space is once more available in the output queue.

On Linux, we can call *ioctl(fd, FIONREAD, &cnt)* to obtain the number of unread bytes in the input queue of the terminal referred to by the file descriptor *fd*. This feature is not specified in SUSv3.



# 64

## PSEUDOTERMINALS

A *pseudoterminal* is a virtual device that provides an IPC channel. On one end of the channel is a program that expects to be connected to a terminal device. On the other end is a program that drives the terminal-oriented program by using the channel to send it input and read its output.

This chapter describes the use of pseudoterminals, showing how they are employed in applications such as terminal emulators, the *script(1)* program, and programs such as *ssh*, which provide network login services.

### 64.1 Overview

Figure 64-1 illustrates one of the problems that pseudoterminals help us solve: how can we enable a user on one host to operate a terminal-oriented program (e.g., *vi*) on another host connected via a network?

As shown in the diagram, by permitting communication over a network, sockets provide part of the machinery needed to solve this problem. However, we can't connect the standard input, output, and error of a terminal-oriented program directly to a socket. This is because a terminal-oriented program expects to be connected to a terminal—to be able to perform the terminal-oriented operations described in Chapters 34 and 62. Such operations include placing the terminal in noncanonical mode, turning echoing on and off, and setting the terminal foreground process group. If a program tries to perform these operations on a socket, then the relevant system calls will fail.

Furthermore, a terminal-oriented program expects a terminal driver to perform certain kinds of processing of its input and output. For example, in canonical mode, when the terminal driver sees the end-of-file character (normally *Control-D*) at the start of a line, it causes the next *read()* to return no data.

Finally, a terminal-oriented program must have a controlling terminal. This allows the program to obtain a file descriptor for the controlling terminal by opening */dev/tty*, and also makes it possible to generate job-control and terminal-related signals (e.g., *SIGTSTP*, *SIGTTIN*, and *SIGINT*) for the program.

From this description, it should be clear that the definition of a terminal-oriented program is quite broad. It encompasses a wide range of programs that we would normally run in an interactive terminal session.

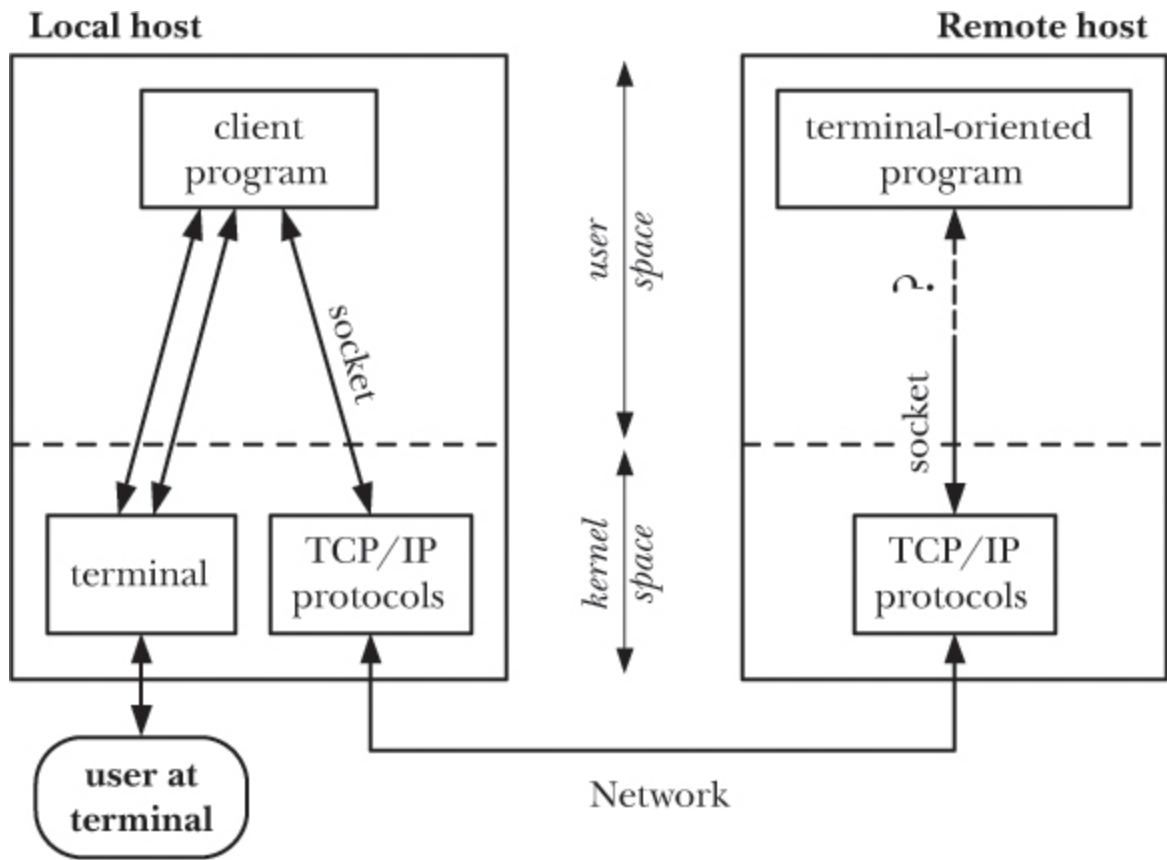


Figure 64-1: The problem: how to operate a terminal-oriented program over a network?

### The pseudoterminal master and slave devices

A pseudoterminal provides the missing link for creating a network connection to a terminal-oriented program. A pseudoterminal is a pair of connected virtual devices: a *pseudoterminal master* and a *pseudoterminal slave*, sometimes jointly referred to as a *pseudoterminal pair*. A pseudoterminal pair provides an IPC channel somewhat like a bidirectional pipe—two processes can open the master and slave and then transfer data in either direction through the pseudoterminal.

The key point about a pseudoterminal is that the slave device appears just like a standard terminal. All of the operations that can be applied to a terminal device can also be applied to a pseudoterminal slave device. Some of these operations aren't meaningful for a pseudoterminal (e.g., setting the terminal line speed or parity), but that's okay, because the pseudoterminal slave silently ignores them.

### How programs use pseudoterminals

Figure 64-2 shows how two programs typically employ a pseudoterminal. (The abbreviation *pty* in this diagram is a commonly used shorthand for *pseudoterminal*, and we employ this abbreviation in various diagrams and function names in this chapter.) The standard input, output, and error of the terminal-oriented program are connected to the pseudoterminal slave, which is also the controlling terminal for the program. On the other side of the pseudoterminal, a driver program acts as a proxy for the user, supplying input to the terminal-oriented program and reading that program's output.

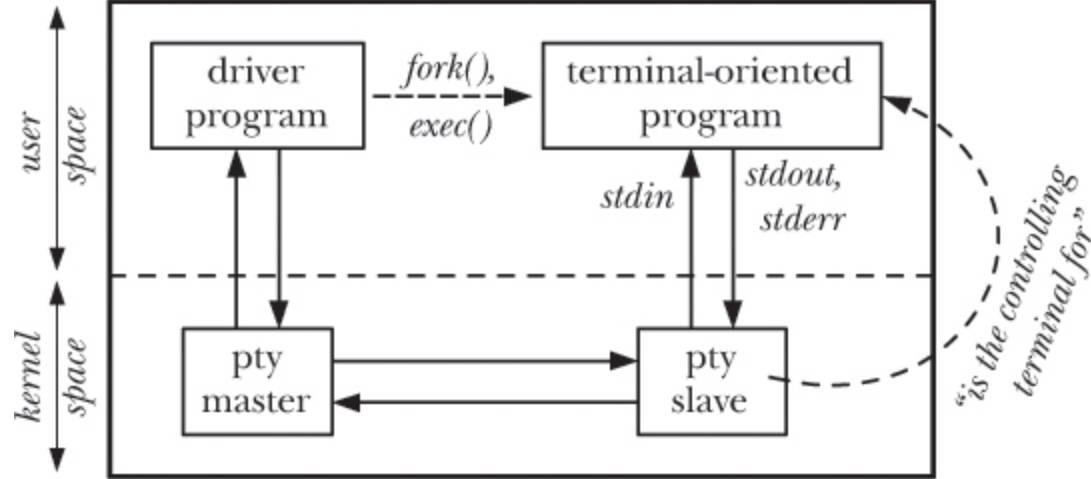


Figure 64-2: Two programs communicating via a pseudoterminal

Typically, the driver program is simultaneously reading from and writing to another I/O channel. It is acting as a relay, passing data in both directions between the pseudoterminal and another program. In order to do this, the driver program must simultaneously monitor input arriving from either direction. Typically, this is done using I/O multiplexing (*select()* or *poll()*), or using a pair of processes or threads to perform data transfer in each direction.

An application that uses a pseudoterminal typically does so as follows:

1. The driver program opens the pseudoterminal master device.
2. The driver program calls *fork()* to create a child process. The child performs the following steps:
  - a) Call *setsid()* to start a new session, of which the child is the session leader (Section 34.3). This step also causes the child to lose its controlling terminal.
  - b) Open the pseudoterminal slave device that corresponds to the master device. Since the child process is a session leader, and it doesn't have a controlling terminal, the pseudoterminal slave becomes the controlling terminal for the child process.
  - c) Use *dup()* (or similar) to duplicate the file descriptor for the slave device on standard input, output, and error.
  - d) Call *exec()* to start the terminal-oriented program that is to be connected to the pseudoterminal slave.

At this point, the two programs can now communicate via the pseudoterminal. Anything that the driver program writes to the master appears as input to the terminal-oriented program on the slave, and anything that the terminal-oriented program writes to the slave can be read by the driver program on the master. We consider further details of pseudoterminal I/O in Section 64.5.

Pseudoterminals can also be used to connect an arbitrary pair of processes (i.e., not necessarily a parent and child). All that is required is that the process that opens the pseudoterminal master informs the other process of the name of the corresponding slave device, perhaps by writing that name to a file or by transmitting it using some other IPC mechanism. (When we use *fork()* in the manner described above, the child automatically inherits sufficient information from the parent to enable it to determine the name of the slave.)

So far, our discussion of the use of pseudoterminals has been abstract. Figure 64-3 shows a specific example: the use of a pseudoterminal by *ssh*, an application that allows a user to securely run a login session on a remote system connected via a network. (In effect, this diagram combines the information from Figure 64-1 and Figure 64-2.) On the remote host, the driver program for the pseudoterminal master is the *ssh* server (*sshd*), and the terminal-oriented program connected to the pseudoterminal slave is the login shell. The *ssh* server is the glue that connects the pseudoterminal via a socket to the *ssh* client. Once all of the details of logging in have been completed, the primary purpose of the *ssh* server and client is to relay characters in either direction between the user's terminal on the local host and the shell on the remote host.

We omit describing many details of the *ssh* client and server. For example, these programs encrypt the data transmitted in either direction across the network. We show a single *ssh* server process on the remote host, but, in fact, the *ssh* server is a concurrent network server. It becomes a daemon and creates a passive TCP socket to listen for incoming connections from *ssh* clients. For each connection, the master *ssh* server forks a child process that handles all of the details for a single client login session. (We refer to this child process as the *ssh* server in Figure 64-3.) Aside from the details of pseudoterminal setup described above, the *ssh* server child authenticates the user, updates the login accounting files on the remote host (as described in Chapter 40), and then execs the login shell.

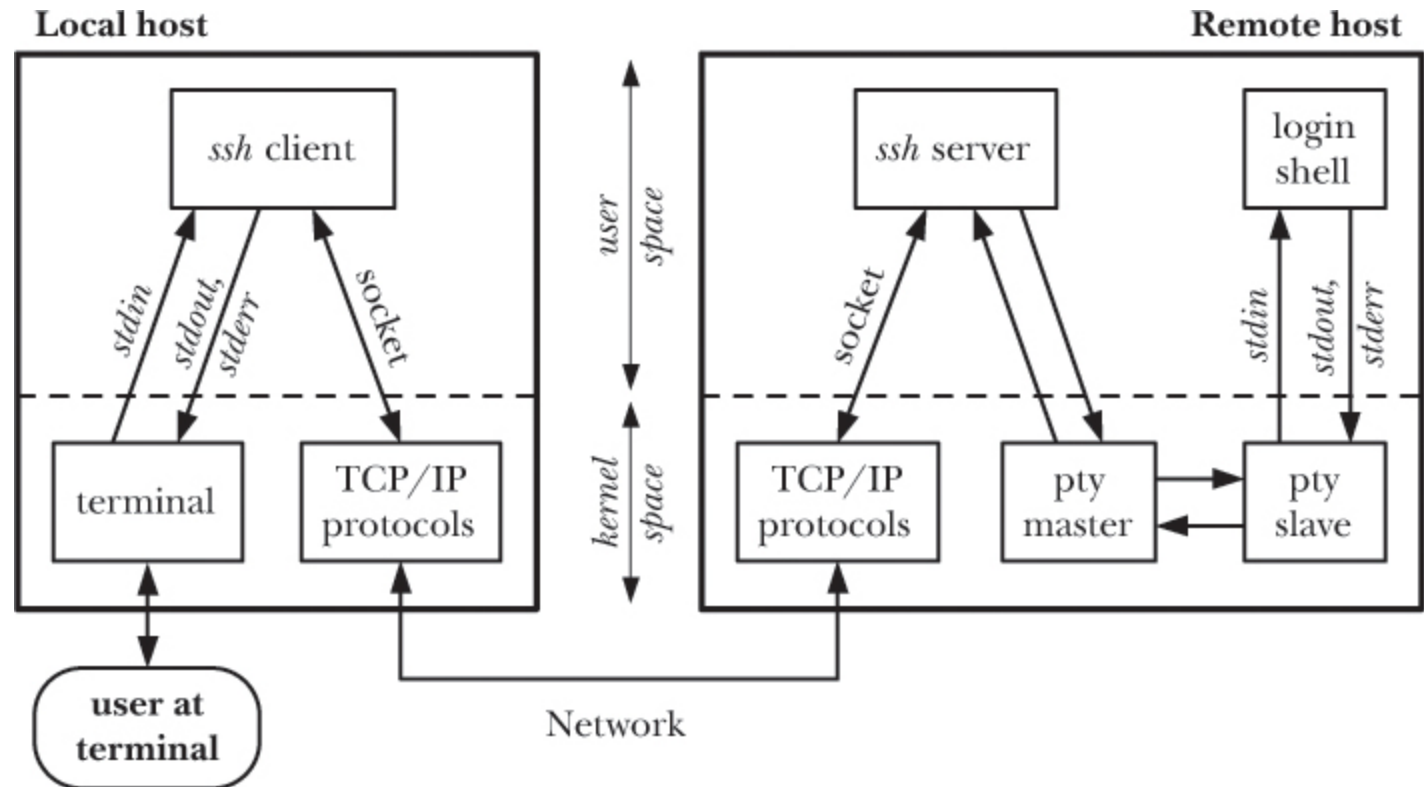


Figure 64-3: How *ssh* uses a pseudoterminal



In some cases, multiple processes may be connected to the slave side of the pseudoterminal. Our *ssh* example illustrates this point. The session leader for the slave is a shell, which creates process groups to execute the commands entered by the remote user. All of these processes have the pseudoterminal slave as their controlling terminal. As with a conventional terminal, one of these process groups can be the foreground process group for the pseudoterminal slave, and only this process group is allowed to read from the slave and (if the `TOSTOP` bit has been set) write to it.

### Applications of pseudoterminals

Pseudoterminals are also used in many applications other than network services. Examples include the following:

- The *expect(1)* program uses a pseudoterminal to allow an interactive terminal-oriented program to be driven from a script file.
- Terminal emulators such as *xterm* employ pseudoterminals to provide the terminal-related functionality that goes with a terminal window.
- The *screen(1)* program uses pseudoterminals to multiplex a single physical terminal (or terminal window) between multiple processes (e.g., multiple shell sessions).
- Pseudoterminals are used in the *script(1)* program, which records all of the input and output that occurs during a shell session.
- Sometimes a pseudoterminal is useful to circumvent the default block buffering performed by the *stdio* functions when writing output to a disk file or pipe, as opposed to the line buffering used for terminal output. (We consider this point further in Exercise 64-7.)

### System V (UNIX 98) and BSD pseudoterminals

BSD and System V provided different interfaces for finding and opening the two halves of a pseudoterminal pair. The BSD pseudoterminal implementation was historically the better known, since it was used with many sockets-based network applications. For compatibility reasons, many UNIX implementations eventually came to support both styles of pseudoterminals.

The System V interface is somewhat simpler to use than the BSD interface, and the SUSv3 specification of pseudoterminals is based on the System V interface. (A pseudoterminal specification first appeared in SUSv1.) For historical reasons, on Linux systems, this type of pseudoterminal is commonly referred to as a *UNIX 98* pseudoterminal, even though the UNIX 98 standard (i.e., SUSv2) required pseudoterminals to be STREAMS-based, and the Linux implementation of pseudoterminals is not. (SUSv3 doesn’t require a STREAMS-based implementation.)

Early versions of Linux supported only BSD-style pseudoterminals, but, since kernel 2.2, Linux has supported both types of pseudoterminals. In this chapter, we focus on UNIX 98 pseudoterminals. We describe the differences for BSD pseudoterminals in Section 64.8.

## 64.2 UNIX 98 Pseudoterminals

Bit by bit, we’ll work toward the development of a function, *ptyFork()*, that does most of the work to create the setup shown in Figure 64-2. We’ll then use this function to implement the *script(1)* program. Before doing this though, we look at the various library functions used with UNIX 98 pseudoterminals:

- The *posix\_openpt()* function opens an unused pseudoterminal master device, returning a file descriptor that is used to refer to the device in later calls.
- The *grantpt()* function changes the ownership and permissions of the slave device corresponding to a pseudoterminal master device.
- The *unlockpt()* function unlocks the slave device corresponding to a pseudoterminal master device, so that the slave device can be opened.
- The *ptsname()* function returns the name of the slave device corresponding to a pseudoterminal master device. The slave device can then be opened using *open()*.

### 64.2.1 Opening an Unused Master: *posix\_openpt()*

The *posix\_openpt()* function finds and opens an unused pseudoterminal master device, and returns a file descriptor that can later be used to refer to this device.

```
#define _XOPEN_SOURCE 600
#include <stdlib.h>
#include <fcntl.h>

int posix_openpt(int flags);
```

Returns file descriptor on success, or -1 on error

The *flags* argument is constructed by ORing zero or more of the following constants together:

`O_RDWR`      Open the device for both reading and writing. Normally, we would always include this constant in *flags*.

`O_NOCTTY`      Don’t make this terminal the controlling terminal for the process. On Linux, a pseudoterminal master can’t become a controlling terminal for a process, regardless of whether the `O_NOCTTY` flag is specified when calling *posix\_openpt()*. (This makes sense because the pseudoterminal master isn’t really a terminal; it is the other side of a terminal to which the slave is connected.) However, on some implementations, `O_NOCTTY` is required if we want to prevent a process from acquiring a controlling terminal as a consequence of opening a pseudoterminal master device.

Like *open()*, *posix\_openpt()* uses the lowest available file descriptor to open the pseudoterminal master.

Calling *posix\_openpt()* also results in the creation of a corresponding pseudoterminal slave device file in the `/dev/pts` directory. We say more about this file when we describe the *ptsname()* function below.

```
/* Child falls through to here */

⑤  if (setsid() == -1)                /* Start a new session */
    err_exit("ptyFork:setsid");

⑥  close(mfd);                      /* Not needed in child */

⑦  slaveFd = open(slname, O_RDWR);   /* Becomes controlling tty */
    if (slaveFd == -1)
        err_exit("ptyFork:open-slave");

⑧  #ifdef TIOCSCTTY                  /* Acquire controlling tty on BSD */
    if (ioctl(slaveFd, TIOCSCTTY, 0) == -1)
        err_exit("ptyFork:ioctl-TIOCSCTTY");
    #endif

⑨  if (slaveTermios != NULL)         /* Set slave tty attributes */
    if (tcsetattr(slaveFd, TCSANOW, slaveTermios) == -1)
        err_exit("ptyFork:tcsetattr");

⑩  if (slaveWS != NULL)              /* Set slave tty window size */
    if (ioctl(slaveFd, TIOCSWINSZ, slaveWS) == -1)
        err_exit("ptyFork:ioctl-TIOCSWINSZ");

/* Duplicate pty slave to be child's stdin, stdout, and stderr */

⑪  if (dup2(slaveFd, STDIN_FILENO) != STDIN_FILENO)
    err_exit("ptyFork:dup2-STDIN_FILENO");
    if (dup2(slaveFd, STDOUT_FILENO) != STDOUT_FILENO)
        err_exit("ptyFork:dup2-STDOUT_FILENO");
    if (dup2(slaveFd, STDERR_FILENO) != STDERR_FILENO)
        err_exit("ptyFork:dup2-STDERR_FILENO");

    if (slaveFd > STDERR_FILENO)      /* Safety check */
        close(slaveFd);              /* No longer need this fd */

    return 0;                        /* Like child of fork() */
}
```

pty/pty\_fork.c

## 64.5 Pseudoterminal I/O

A pseudoterminal pair is similar to a bidirectional pipe. Anything that is written on the master appears as input on the slave, and anything that is written on the slave appears as input on the master.

The point that distinguishes a pseudoterminal pair from a bidirectional pipe is that the slave side operates like a terminal device. The slave interprets input in the same way as a normal controlling terminal would interpret keyboard input. For example, if we write a *Control-C* character (the usual terminal *interrupt* character) to the pseudoterminal master, the slave will generate a `SIGINT` signal for its foreground process group. Just as with a conventional terminal, when a pseudoterminal slave operates in canonical mode (the default), input is buffered line by line. In other words, the program reading from the pseudoterminal slave will see (a line of) input only when we write a newline character to the pseudoterminal master.

Like pipes, pseudoterminals have a limited capacity. If we exhaust this capacity, then further writes are blocked until the process on the other side of the pseudoterminal has consumed some bytes.

On Linux, the pseudoterminal capacity is about 4 kB in each direction.

If we close all file descriptors referring to the pseudoterminal master, then:

- If the slave device has a controlling process, a `SIGHUP` signal is sent to that process (see Section 34.6).
- A `read()` from the slave device returns end-of-file (0).
- A `write()` to the slave device fails with the error `EIO`. (On some other UNIX implementations, `write()` fails with the error `ENXIO` in this case.)

If we close all file descriptors referring to the pseudoterminal slave, then:

- A `read()` from the master device fails with the error `EIO`. (On some other UNIX implementations, a `read()` returns end-of-file in this case.)
- A `write()` to the master device succeeds, unless the input queue of the slave device is full, in which case the `write()` blocks. If the slave device is subsequently reopened, these bytes can be read.

UNIX implementations vary widely in their behavior for the last case. On some UNIX implementations, `write()` fails with the error `EIO`. On other implementations, `write()` succeeds, but the output bytes are discarded (i.e., they can’t be read if the slave is reopened). In general, these variations don’t present a problem. Normally, the process on the master side detects that the slave has been closed because a `read()` from the master returns end-of-file or fails. At this point, the process performs no further writes to the master.

### Packet mode

*Packet mode* is a mechanism that allows the process running above a pseudoterminal master to be informed when the following events related to software flow control occur on the pseudoterminal slave:

- the input or output queue is flushed;
- terminal output is stopped or started (*Control-S/Control-Q*); or
- flow control was enabled or disabled.

Packet mode helps with handling software flow control in certain pseudoterminal applications that provide network login services (e.g., *telnet* and *rlogin*).

Packet mode is enabled by applying the `ioctl()` `TIOCPKT` operation to the file descriptor referring to the pseudoterminal master:

```
int arg;

arg = 1;                /* 1 == enable; 0 == disable */
```

```
if (ioctl(mfd, TIOCPKT, &arg) == -1)
    errExit("ioctl");
```

When packet mode is in operation, reads from the pseudoterminal master return either a single nonzero control byte, which is a bit mask indicating the state change(s) that occurred on the slave device, or a 0 byte followed by one or more bytes of data that were written on the pseudoterminal slave.

When a state change occurs on a pseudoterminal that is operating in packet mode, *select()* indicates that an exceptional condition (the *exceptfds* argument) has occurred on the master, and *poll()* returns *POLLPRI* in the *revents* field. (Refer to Chapter 63 for descriptions of *select()* and *poll()*.)

Packet mode is not standardized in SUSv3, and some details vary on other UNIX implementations. Further details of packet mode on Linux, including the bit-mask values used to indicate state changes, can be found in the *tty\_ioctl(4)* manual page.

## 64.6 Implementing *script(1)*

We are now ready to implement a simple version of the standard *script(1)* program. This program starts a new shell session, and records all input and output from the session to a file. Most of the shell sessions shown in this book were recorded using *script*.

In a normal login session, the shell is connected directly to the user's terminal. When we run *script*, it places itself between the user's terminal and the shell, and uses a pseudoterminal pair to create a communication channel between itself and the shell (see Figure 64-4). The shell is connected to the pseudoterminal slave. The *script* process is connected to the pseudoterminal master. The *script* process acts as a proxy for the user, taking input entered at the terminal and writing it to the pseudoterminal master, and reading output from the pseudoterminal master and writing it to the user's terminal.

In addition, *script* produces an output file (named *typescript* by default) that contains a copy of all bytes that are output on the pseudoterminal master. This has the effect of recording not only the output produced by the shell session, but also the input that is supplied to it. The input is recorded because, just as with a conventional terminal device, the kernel echoes input characters by copying them to the terminal output queue (see Figure 62-1, on page 1291). However, when terminal echoing is disabled, as is done by programs that read passwords, the pseudoterminal slave input is not copied to the slave output queue, and thus is not copied to the script output file.

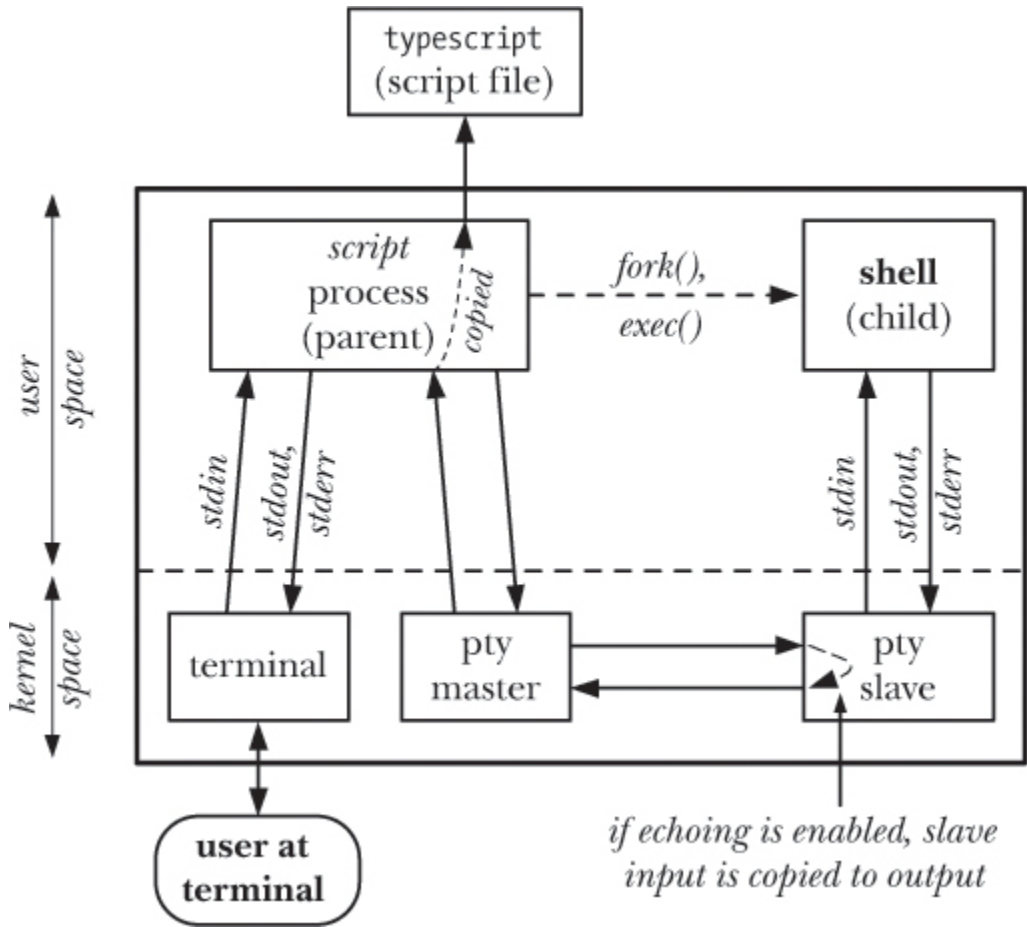


Figure 64-4: The *script* program

Our implementation of *script* is shown in Listing 64-3. This program performs the following steps:

- Retrieve the attributes and window size of the terminal under which the program is run ①. These are passed to the subsequent call to *ptyFork()*, which uses them to set the corresponding values for the pseudoterminal slave device.
- Call our *ptyFork()* function (Listing 64-2) to create a child process that is connected to the parent via a pseudoterminal pair ②.
- After the *ptyFork()* call, the child execs a shell ④. The choice of shell is determined by the setting of the *SHELL* environment variable ③. If the *SHELL* variable is not set or its value is an empty string, then the child execs */bin/sh*.
- After the *ptyFork()* call, the parent performs the following steps:
  - Open the output script file ⑤. If a command-line argument is supplied, this is used as the name of the script file. If no command-line argument is supplied, the default name *typescript* is used.
  - Place the terminal in raw mode (using the *ttySetRaw()* function shown in Listing 62-3, on page 1310), so that all input characters are passed directly to the *script* program without being modified by the terminal driver ⑥. Characters output by the *script* program are likewise not modified by the terminal driver.

The fact that the terminal is in raw mode doesn't mean that raw, uninterpreted control characters will be transmitted to the shell, or whatever other process group is in the foreground for the pseudoterminal slave device, nor that output from that process group is passed raw to the user's terminal. Instead, interpretation of terminal special characters is taking place within the slave device (unless the slave was also explicitly placed in raw mode by an application). By placing the user's terminal in raw mode, we prevent a *second* round of interpretation of input and output characters from occurring.